Assignment 7:
Drones: Bridge + Template Method + Environment + Strategy + Observer + Factory + CoR + API

Design Patterns

Goal

You will design and implement from scratch a modular drone mission framework where **mission algorithms** use the **Template Method**, movement/platform concerns are decoupled via the **Bridge**, missions react to **dynamic environment feedback**, and several additional patterns (**Strategy**, **Factory**, **Observer**, **Chain of Responsibility**) make the system robust and extensible — plus a minimal HTTP API to orchestrate missions.

1 Overview & Motivation

This assignment blends architecture, simulation, and multi-pattern design:

- **Template Method**: Mission workflow (load config → analyze environment → preflight → navigate → act → react → collect → return → postprocess).
- **Bridge**: Mission logic (abstraction) decoupled from movement/platform implementations (Air, Sea, Surface).
- **Strategy**: Environment reaction behaviors (wind, waves, cracks).
- **Factory Method / Abstract Factory**: Runtime composition of missions + environments + platforms.
- **Observer**: Environment events pushed to missions.
- **Chain of Responsibility**: Fail-safe handlers.
- **API**: Minimal HTTP interface for running missions.

2 Requirements

1. Implement the whole system from scratch.
2. Implement missions:
   `SeaExploration`, `Agriculture`, `DefectsDetection`, +2 additional missions.
3. Implement platforms: `AirPlatform`, `SeaPlatform`, `SurfacePlatform`.
4. Implement environments: `AirEnvironment`, `SeaEnvironment`, `SurfaceEnvironment`.
5. Implement all required patterns.
6. Provide HTTP API endpoints:

   ```
   POST /mission/run
   GET  /mission/status/{id}
   GET  /mission/result/{id}
   ```

7. Provide unit tests, an integration test, and a demo script.
8. Provide README, UML diagrams, and a short design document.

3 High-Level Architecture

```
API (FastAPI) -> Factory -> {Mission (Template)
                             + Controller (Bridge)
                             + Environment
                             + Strategy
                             + CoR}
                      ↑
          MovementImplementor (Air/Sea/Surface)
                      ↑
Environment (Observer publishes events) -> Missions
                      ↑
               ReactionStrategy
                      ↑
               FailSafeChain (CoR)
```

4 Template Method: Mission Algorithm

Each mission extends `DroneMission`, which implements:

1. load_config()

2. setup_event_subscriptions()
3. analyze_environment()
4. preflight_check()
5. navigate_to_area()
6. perform_payload_action()
7. **while environment_requires_reaction():**
   react_to_environment() via Strategy
8. collect_and_store_data()
9. return_to_base()
10. postprocess_results()

Missions must only use movement via the `DroneController` (Bridge).

5 Bridge Pattern: Movement Layer

MovementImplementor Interface (Sketch)

```
class MovementImplementor(ABC):
    def takeoff(self): ...
    def land(self): ...
    def move_to(self, coord): ...
    def adjust_course(self, vector): ...
    def hold_position(self): ...
    def set_mode(self, mode): ...       # single / swarm
    def broadcast(self, message): ...   # swarm comms
```

Controller Abstraction

```
class DroneController:
    def goto(self, coord, retries=3): ...
    def adjust_course(self, vector): ...
    def set_swarm(self): ...
    def set_single(self): ...
```

6 Environment Models & Observer Pattern

Environments produce readings and fire events.

```
class Environment(ABC):
    def sample(self) -> dict: ...
    def subscribe(self, callback): ...
    def start(self): ...
```

Examples of readings:

- Air: wind_speed, visibility, temperature
- Sea: wave_height, current_speed, salinity
- Surface: terrain_roughness, obstacle_density

Environments publish `EnvironmentEvent` objects via an `EventBus`.

7 Mission Config

```
@dataclass
class MissionConfig:
    mission_id: str
    mission_type: str
    environment_type: str
    platform_type: str
    mode: str
    target_area: Coord
    base_area: Coord
    thresholds: dict
    behavior_params: dict
```

8 Strategy Pattern: Reactions

```
class ReactionStrategy(ABC):
    def react(self, mission, reading): ...
```

Examples:

- WaveReaction
- WindReaction
- CrackReaction

9 Chain of Responsibility: Fail-Safe

Implement handlers:

- ReRouteHandler
- AdjustAltitudeHandler
- SwarmReassignHandler
- EmergencyLandHandler

Each handler either resolves the issue or forwards to the next.

10 Abstract Factory / Factory Method

`MissionFactory` must create:

- appropriate Environment
- appropriate MovementImplementor
- DroneController
- concrete Mission subclass
- appropriate ReactionStrategy
- CoR fail-safe chain

11 HTTP API Specification

Provide endpoints:

```
POST /mission/run
GET  /mission/status/{id}
GET  /mission/result/{id}
```

Example:

```
@app.post("/mission/run")
def run_mission(cfg):
    mission = factory.create_from_dict(cfg)
    result = mission.execute_mission()
    return {"mission_id": mission.config.mission_id,
            "result": result}
```

12 Project Structure

```
drone_patterns_lab/

 pyproject.toml  # or setup.cfg
 README.md

 drones/
    __init__.py
    api/
       __init__.py
       server.py
       endpoints.py

    factory/
       __init__.py
       mission_factory.py
       config_loader.py

    observer/
       __init__.py
       event_bus.py
       events.py

    cor/
       __init__.py
       base.py
       reroute_handler.py
       adjust_altitude_handler.py
       swarm_reassign_handler.py
       emergency_land_handler.py

    strategy/
       __init__.py
       base.py
       wave_reaction.py
       wind_reaction.py
       crack_reaction.py

    bridge/
       __init__.py
       base.py
       controller.py
       air.py
       sea.py
       surface.py

    template/
```

```
        __init__.py
        base.py

    missions/
        __init__.py
        sea_exploration.py
        agriculture.py
        defects_detection.py
        rescue.py
        pollution_monitoring.py

    environment/
        __init__.py
        base.py
        air_env.py
        sea_env.py
        surface_env.py

    config/
        __init__.py
        mission_config.py

    utils/
         __init__.py
         logger.py
         telemetry.py
         persistence.py
         math_utils.py

tests/
    unit/
        test_bridge.py
        test_template.py
        test_strategy.py
        test_cor.py
    integration/
         test_mission_end_to_end.py
docs/
     design.md
     UML.png
```

13 Package Responsibilities

- **api/** — HTTP API.
- **factory/** — MissionFactory + config loader.

- **observer/** — event bus + event definitions.
- **cor/** — fail-safe CoR handlers.
- **strategy/** — reaction strategies.
- **bridge/** — movement implementors + controller.
- **template/** — Template Method base class.
- **missions/** — concrete missions.
- **environment/** — environment simulation models.
- **config/** — mission configuration objects.
- **utils/** — logging, telemetry, persistence.

14 Testing Requirements

Unit Tests

- Template Method step order
- Bridge delegation correctness
- Strategy behavior correctness
- Observer event propagation
- CoR failure handling

Integration Test

Full mission run composed by factory, including persistence.

15 Required Deliverables

- full Python implementation
- README
- demo script
- UML diagrams and design doc
- unit & integration tests
- API interface

16 Helpful Hints

- Avoid `isinstance()` in mission logic.
- Use mocks/fakes for deterministic unit testing.
- Swarm logic may be simulated (no real networking).
- API may be synchronous for simplicity.